

Error Correction Plugin

How to integrate the library into WordPress sites — three paths from the simplest to the most formal, plus WordPress-specific caveats.

Short answer

Yes, it works in WordPress — but it needs a bit of “plumbing”. This is *not* a native WordPress plugin (no PHP, no admin panel, no Gutenberg blocks). It is a **client-side library** (pure CSS + JS) that you load on the page.

In WordPress, that plugs in three ways, from simplest to most formal — each described below.

Path 1 — via “Insert Headers and Footers” (zero code)

Install a plugin like **WPCode**, **Insert Headers and Footers**, or **Code Snippets**. Paste the plugin's CSS (between ▼ PLUGIN START ▼ and ▲ PLUGIN CSS END ▲) into a **header** snippet, wrapped in a `<style>` tag. Paste the JS into the **footer**, wrapped in a `<script>` tag. Then, in your articles, use Gutenberg's “**Custom HTML**” block to paste `<div data-ec-block ...>`, `<div data-ec-response ...>`, etc.

This is the fastest path for an editorial pilot — no PHP or theme access required.

Path 2 — embed in the theme

If you (or someone in IT) has access to a child theme, save two files — `ec.css` and `ec.js` — inside the theme folder, and enqueue them via `functions.php`:

```
add_action('wp_enqueue_scripts', function () {
    wp_enqueue_style(
        'ec',
        get_stylesheet_directory_uri() . '/ec.css'
    );
    wp_enqueue_script(
        'ec',
        get_stylesheet_directory_uri() . '/ec.js',
        [], null, true
    );
});
```

The `data-ec-*` attributes still go inside the article body (via Gutenberg's “Custom HTML” block).

Path 3 — package as a real WordPress plugin

Wrapping this in about 15 lines of PHP turns the library into a plugin installable through the “Plugins → Add New” menu:

```
<?php
/**
```

```
* Plugin Name: Error Correction
* Description: Marks journalistic corrections, deletions,
*              and updates.
* Version: 1.0
*/
add_action('wp_enqueue_scripts', function () {
    wp_enqueue_style(
        'ec',
        plugin_dir_url(__FILE__) . 'ec.css'
    );
    wp_enqueue_script(
        'ec',
        plugin_dir_url(__FILE__) . 'ec.js',
        [], null, true
    );
});
```

Ship it as a zip together with `ec.css`, `ec.js`, and the PHP header — and it becomes an official WordPress plugin, installable from the admin panel.

WordPress-specific caveats

1. wpautop (paragraph auto-format)

The `wpautop` function automatically wraps paragraphs in `<p>` and occasionally rearranges loose HTML. The plugin's `<div data-ec-*>` blocks are self-contained and should not be affected, but it is safer to always paste them inside Gutenberg's **Custom HTML** block, which preserves markup as typed.

2. Cache plugins

Plugins like **WP Rocket**, **W3 Total Cache**, and **LiteSpeed Cache** may minify JS/CSS. The plugin uses nothing exotic, so it runs fine with any standard minifier.

3. Theme conflicts

The `ec-* namespace` avoids conflicts with most themes. If a theme does interfere, it is likely inside `<details>/<summary>` or `<blockquote>` — a quick check in the inspector resolves it.

4. Gutenberg quirks

The block editor sometimes “cleans up” unknown HTML attributes inside standard blocks (paragraph, heading, etc.). To keep `data-ec-*` attributes intact, always use the **“Custom HTML”** block — it is the only one that preserves arbitrary HTML. Alternatively, the Classic Editor plugin also works.

What is missing for a serious newsroom rollout

Non-technical editors will not hand-write `data-ec-response`. For production use, the right next step is to build, on top of this library, **custom Gutenberg blocks** — one for each pattern:

- **“Report correction”** block (factual / spelling / imprecision / statement)
- **“Right of reply”** block
- **“Deleted excerpt”** block
- **“New content”** block

Each block renders a small form inside the editor (with fields for *reason*, *before/after*, *date*, etc.) and outputs the correct HTML automatically. It requires more implementation work — including a `@wordpress/scripts` build setup — but it is what unlocks editorial adoption at scale.

Summary

Path	Who does it	When to use
1. Insert Headers and Footers	Site editor / SEO	Quick pilot, no code
2. Embed in child theme	Dev / IT	Own site, full control

3. Packaged WordPress plugin	Dev (15 lines of PHP)	Distribute across sites
4. Custom Gutenberg blocks	Dev (build setup)	Editorial use in production

What is ready in this prototype: paths 1, 2, and 3 work as described — just copy and paste the CSS/JS from `error-correction-plugin.html` (or the separate `en/ec.css` and `en/ec.js` files in this repository). **What still requires development:** path 4 (Gutenberg blocks), the next step for an editorial rollout at scale.

*Document generated from information discussed during the development of the research prototype —
Lívia Vieira, 2026.*